Pre-entrega Módulo 6
Circuito cerrado conversacional de voz (Voice AI)

Proyecto integrador **AgendaBot** — AI Automation Avanzado (Coderhouse)

Julio Martín Waisburd

11/10/2026

Índice

1. Viabilidad de la voz en AgendaBot (ROI y fatiga cognitiva) ..... pág. 2
2. Arquitectura del circuito cerrado de voz ..... pág. 3
3. Interceptación binaria, oídos (Whisper) y cerebro (AI Agent) ..... pág. 5
4. Contención financiera: máximo 200 caracteres ..... pág. 7
5. Voz del agente (ElevenLabs) y salida multimedia ..... pág. 8
6. Ruta de contingencia no-code ..... pág. 10
7. Compliance: audios en memoria volátil y destrucción ..... pág. 12
8. Pruebas de punta a punta y hallazgos ..... pág. 13
9. Anexo: configuración exportada en formato texto ..... pág. 14

1. Viabilidad de la voz en AgendaBot (ROI y fatiga cognitiva)

Diagnóstico: la voz conviene en AgendaBot para pedidos **cortos y de manos libres** (sacar, confirmar o consultar un turno; una pregunta frecuente) y **no** conviene cuando el paciente necesita ver o releer datos (listas largas, emails, números). Por eso la capa de voz responde con audio solo lo breve y deja lo largo o lo exacto por escrito.

Proceso de AgendaBot	¿Voz?	Por qué
Sacar o reprogramar un turno ("agendame una tomografía el 11 a las 21:30")	**Sí**	Una frase de 5–8 s reemplaza varios mensajes tipeados; la confirmación entra en un audio de ~6 s.
Consultar el propio turno o una pregunta frecuente	**Sí**	Respuesta de una o dos oraciones: el caso de uso de alto retorno de la unidad.
Listado completo de profesionales (8 nombres en Cardiología + Infantil)	**No**	Escuchar ocho nombres genera fatiga cognitiva; el audio da los primeros y el resto va por texto.
Dictar un email	**No**	Whisper lo transcribe mal (ver pieza 6): se pide por escrito.
Pregunta fuera de alcance (dólar, trámites)	Audio corto	Una frase fija de límite honesto (106 caracteres).

Costo por interacción (`Costo = Whisper (min) + LLM (tokens) + ElevenLabs (caracteres)`), medido en las pruebas:

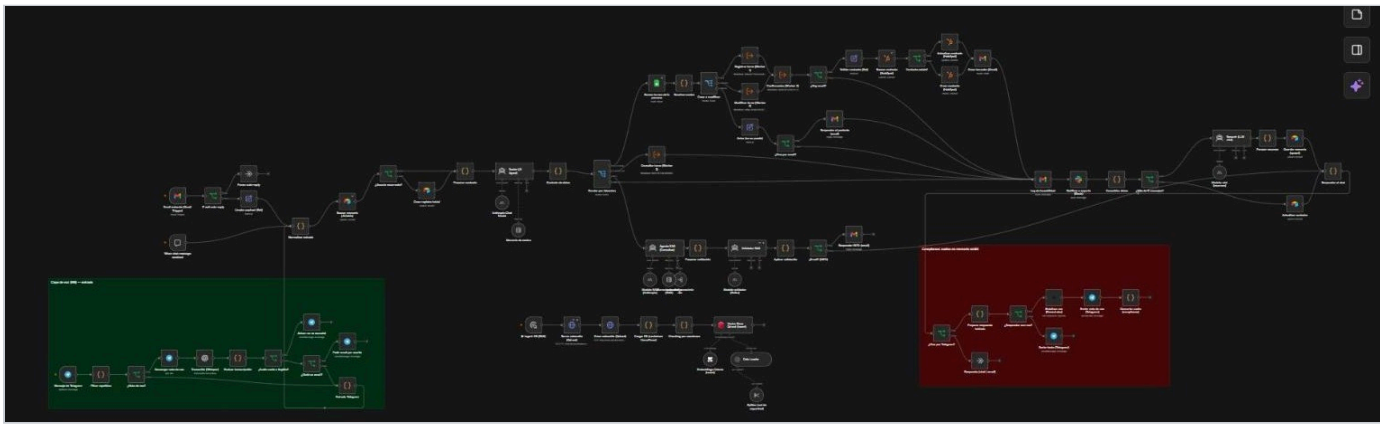
Componente	Unidad	Medido en las pruebas	Contención aplicada
OpenAI Whisper	minuto de audio	notas de 3–8 s (≈ US$ 0,0003–0,0008 a US$ 0,006/min)	idioma fijo `es` (no detecta idioma)
Claude (Router + Agente RAG + validador)	tokens	~6.700 tokens en una consulta INFO	prompts acotados; mismo pipeline que el M5
ElevenLabs	carácter sintetizado	87–134 caracteres por respuesta	**tope de 200 caracteres** (prompt + código), sin citas, URLs ni emojis

El componente que más crece con el uso es ElevenLabs: sin el tope, una respuesta de texto del M5 (lista de profesionales con citas) supera los 600 caracteres; con el tope queda en ~130, es decir, **un ahorro de más del 75 % por respuesta hablada**.

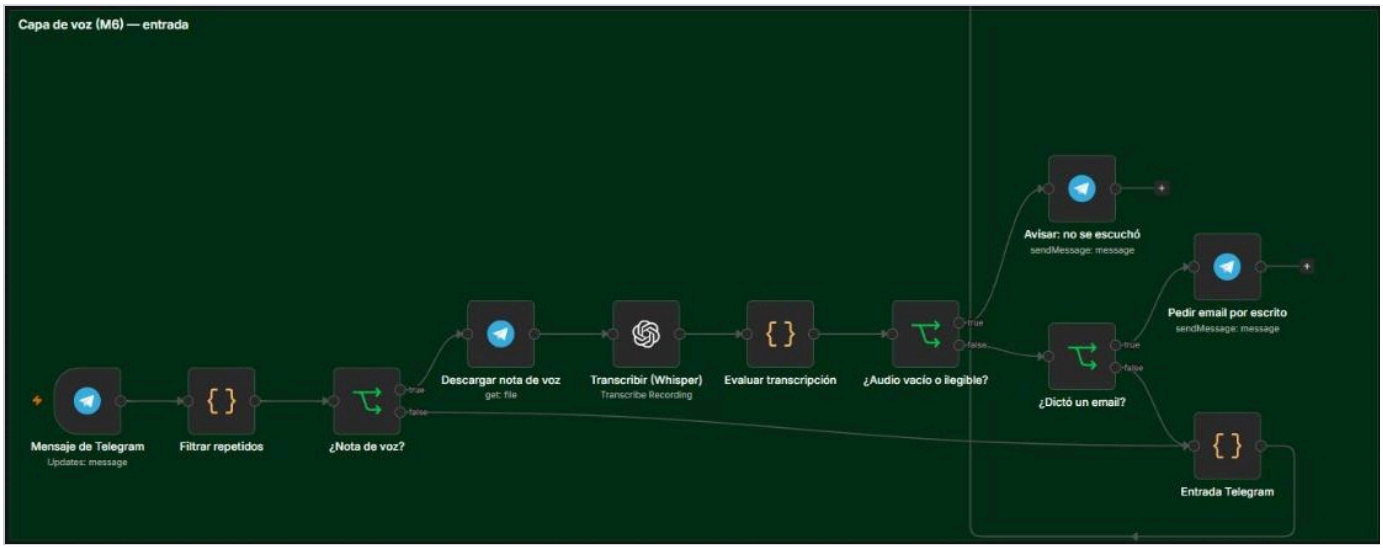
2. Arquitectura del circuito cerrado de voz

El M6 **no arranca de cero**: es el Manager del M5 (multi-agente + memoria + integraciones + RAG con validador) duplicado y apuntado a los recursos 06 (`Agenda_Turnos (06)`, tabla `Memoria_M6`), al que se le sumó **Telegram como un canal más**, junto al chat y al email. El circuito es cerrado: entra una nota de voz y sale una nota de voz.

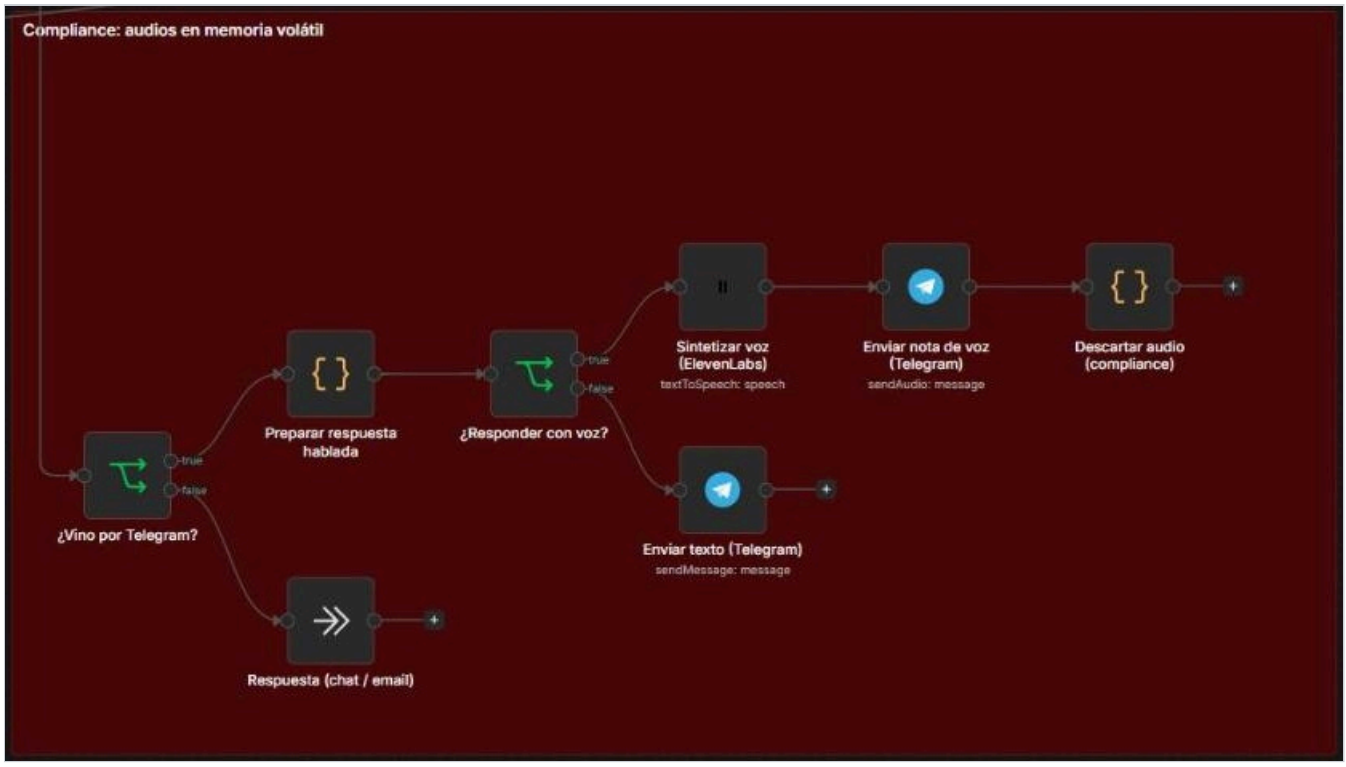
```
Telegram → filtro de repetidos → ¿nota de voz? → descargar audio → Whisper (data, es)
   → contingencia (vacío / alucinación / email dictado) → Manager M5 (Router → workers o RAG + validador)
   → respuesta hablada (≤ 200 car.) → ElevenLabs (Multilingual v2) → Telegram Send Audio → descartar audio
```



Entrada de voz (zona verde): trigger de Telegram, Whisper y la ruta de contingencia; después el mensaje entra al Manager como canal `telegram_voz`.

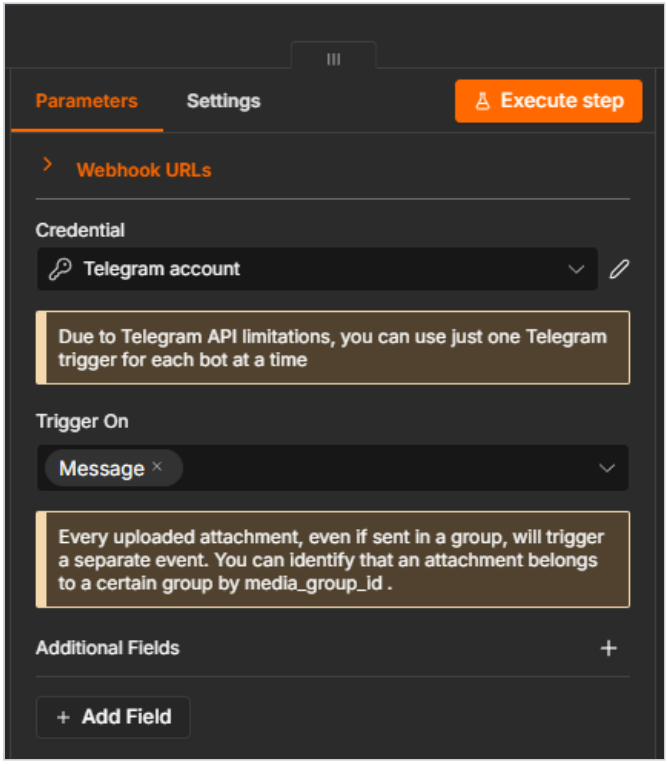


Salida de voz (zona roja): si el pedido vino por Telegram, la respuesta se prepara para el oído, se sintetiza y se envía como audio; si vino por chat o email, sigue como en el M5.

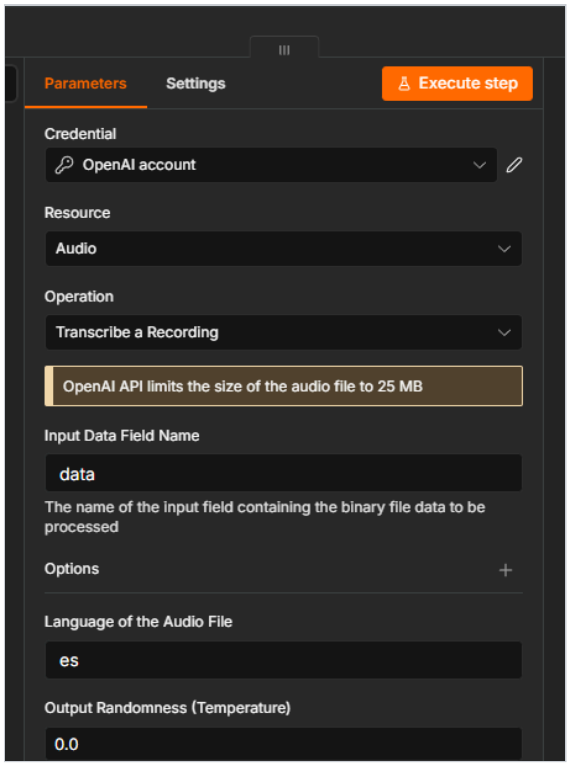

Compliance: audios en memoria volátil
¿Vino por Telegram?
true
false
Preparar respuesta hablada
¿Responder con voz?
true
false
Sintetizar voz (ElevenLabs)
textToSpeech: speech
Enviar nota de voz (Telegram)
sendAudio: message
Descartar audio (compliance)
Enviar texto (Telegram)
sendMessage: message
Respuesta (chat / email)

3. Interceptación binaria, oídos (Whisper) y cerebro (AI Agent)

Trigger de Telegram. Un bot oficial creado con @BotFather; el nodo *Telegram Trigger* escucha `message` y recibe las notas de voz en tiempo real. n8n corre en local y se expone a Telegram con un túnel HTTPS de Cloudflare (`WEBHOOK_URL` apunta al túnel), el mismo esquema que mostró la demo de la clase.



Whisper. Telegram no entrega el audio dentro del evento sino un `file_id`: por eso, entre el trigger y Whisper, hay un IF que separa voz de texto y un *Telegram › Get File* que descarga la nota de voz como binario `data` (igual que en la demo de la clase). El nodo *OpenAI › Audio › Transcribe a Recording* tiene mapeado **`data`** en *Input Data Field Name* y el idioma fijado en **`es`**, para que no gaste tiempo detectando el idioma. Temperatura 0.

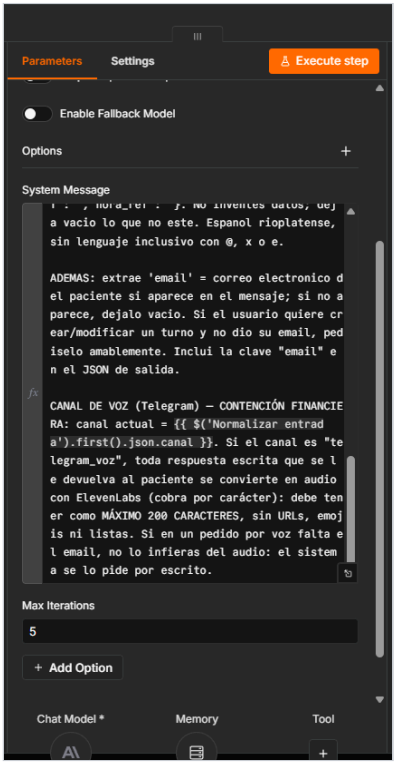


Cerebro. El texto transcrito entra al Manager por "Normalizar entrada" como canal `telegram_voz` y llega al **Router (AI Agent)**: el mismo nodo agéntico del M5 (AI Agent v3.1, que en esta versión de n8n trabaja como *Tools Agent*), con memoria de Airtable, que deriva a los workers de turnos o al Agente RAG. Así, una nota de voz puede sacar un turno o consultar la base documental igual que un mensaje escrito.

4. Contención financiera: máximo 200 caracteres

La regla está **escrita en el System Message** del agente central (Router) y del Agente RAG, que es el que redacta las respuestas informativas. Se activa cuando el canal es `telegram_voz`:

> CANAL DE VOZ (Telegram) — CONTENCIÓN FINANCIERA: [...] toda respuesta escrita que se le devuelva al paciente se convierte en audio con ElevenLabs (cobra por carácter): debe tener como **MÁXIMO 200 CARACTERES**, sin URLs, emojis ni listas.



En el Agente RAG la regla agrega pautas de diseño para el oído de la unidad 6: frases cortas (sujeto + verbo + predicado), puntos seguidos para marcar pausas y, si el dato es una lista larga, nombrar los primeros y ofrecer el resto por escrito.

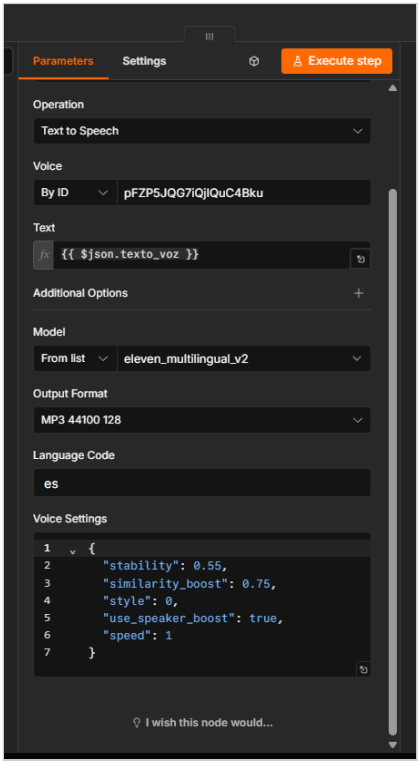
Segunda barrera en código. El nodo "Preparar respuesta hablada" saca del texto lo que no debe leerse en voz alta (citas `[Fuente: …]`, URLs, markdown, viñetas, emojis) y, si aun así pasa de 200 caracteres, corta en el último fin de oración. Así el tope se cumple aunque el modelo no lo respete.

Prueba	Caracteres enviados a ElevenLabs
Confirmación de turno (tomografía)	87
Profesionales de Cardiología (RAG)	134
Pregunta fuera de alcance	106

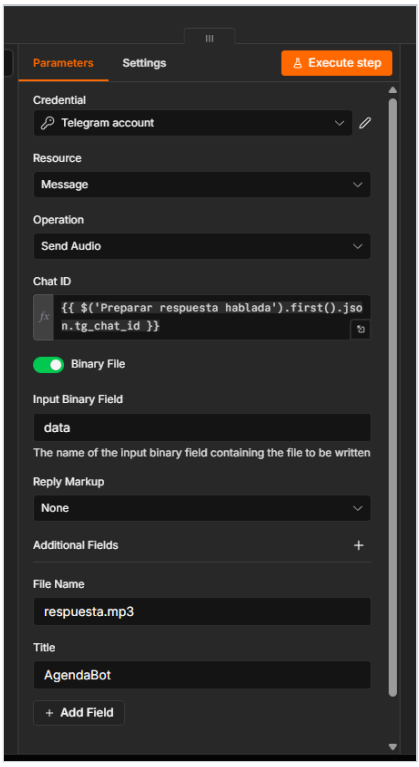
5. Voz del agente (ElevenLabs) y salida multimedia

ElevenLabs. Se usa el nodo oficial de ElevenLabs para n8n (paquete `@elevenlabs/n8n-nodes-elevenlabs`, publicado por ElevenLabs; n8n 2.39 no lo trae de fábrica), operación *Speech › Text to Speech*, con el texto `{{ $json.texto_voz }}` que dejó la pieza 4.

- **Modelo:** `eleven_multilingual_v2` (Eleven Multilingual), idioma `es`.
- **Stability 0,55 y Clarity (similarity_boost) 0,75:** perfil de **voz de soporte** de la guía de la unidad 2 (estabilidad media, tono cálido) para una clínica. En este nodo los controles no son sliders: se cargan en el JSON *Voice Settings* con los mismos valores.
- **Formato:** MP3 44,1 kHz 128 kbps.



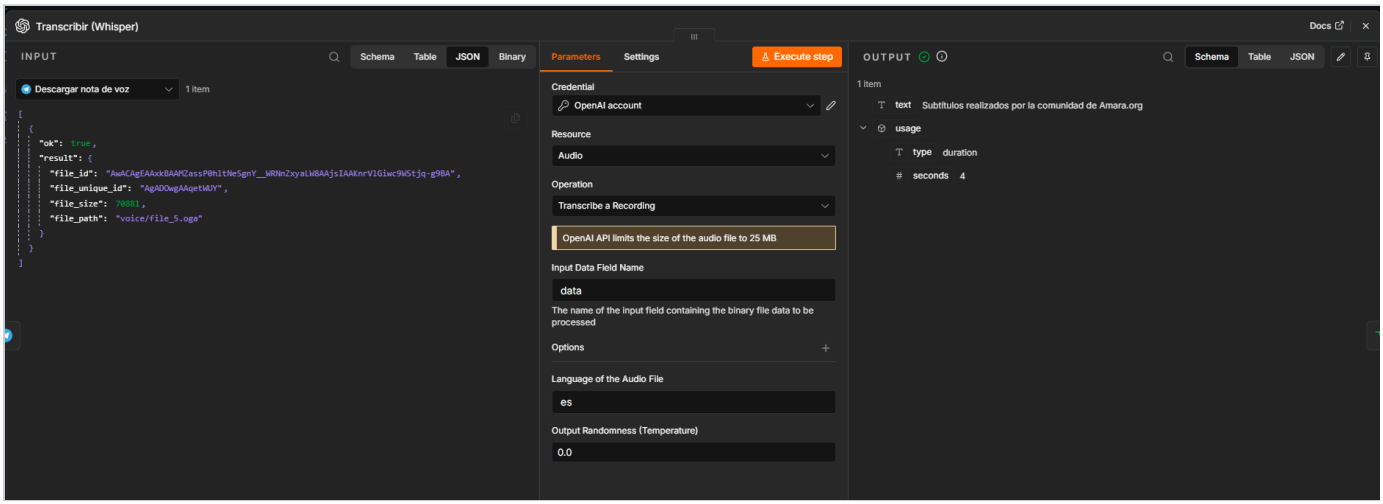
Salida. *Telegram › Send Audio* con *Binary File* activado toma el binario **`data`** que generó ElevenLabs y lo envía al mismo chat del paciente (`tg_chat_id`), como `respuesta.mp3`. En las pruebas, los audios enviados duraron 6 s (confirmación de turno) y 10 s (consulta de Cardiología).



Si el pedido llegó por **texto** de Telegram, la respuesta sale por *Send Message* (texto completo, sin recorte): la voz se reserva para quien habló.

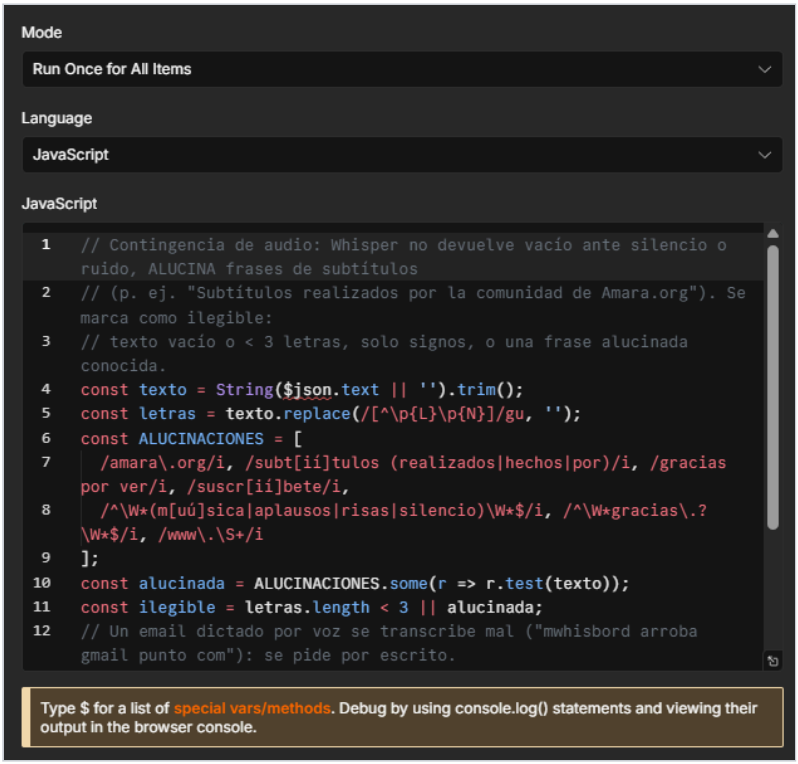
6. Ruta de contingencia no-code

Hallazgo: ante una nota de voz en silencio, Whisper **no devuelve texto vacío: inventa una frase**. En la prueba devolvió "*Subtítulos realizados por la comunidad de Amara.org*", una alucinación típica del modelo (aprendió esas leyendas de subtítulos). Un IF que solo mirara "texto vacío o menor a 3 caracteres", como sugiere el material, la habría dejado pasar al agente.



Por eso, inmediatamente después de Whisper, el nodo **"Evaluar transcripción"** marca el audio como ilegible si:

- el texto está vacío, tiene menos de 3 letras o solo signos;
- coincide con una alucinación conocida de Whisper ("Amara.org", "Subtítulos realizados por…", "Gracias por ver", "[Música]", "Gracias." suelto).



El IF **"¿Audio vacío o ilegible?"** desvía esos casos a un mensaje de texto: "*Perdón, no logré escuchar tu audio con claridad. ¿Podés grabarlo de nuevo en un lugar con menos ruido, o escribirme tu consulta?*". El flujo no se rompe y el agente no responde a una frase fantasma.

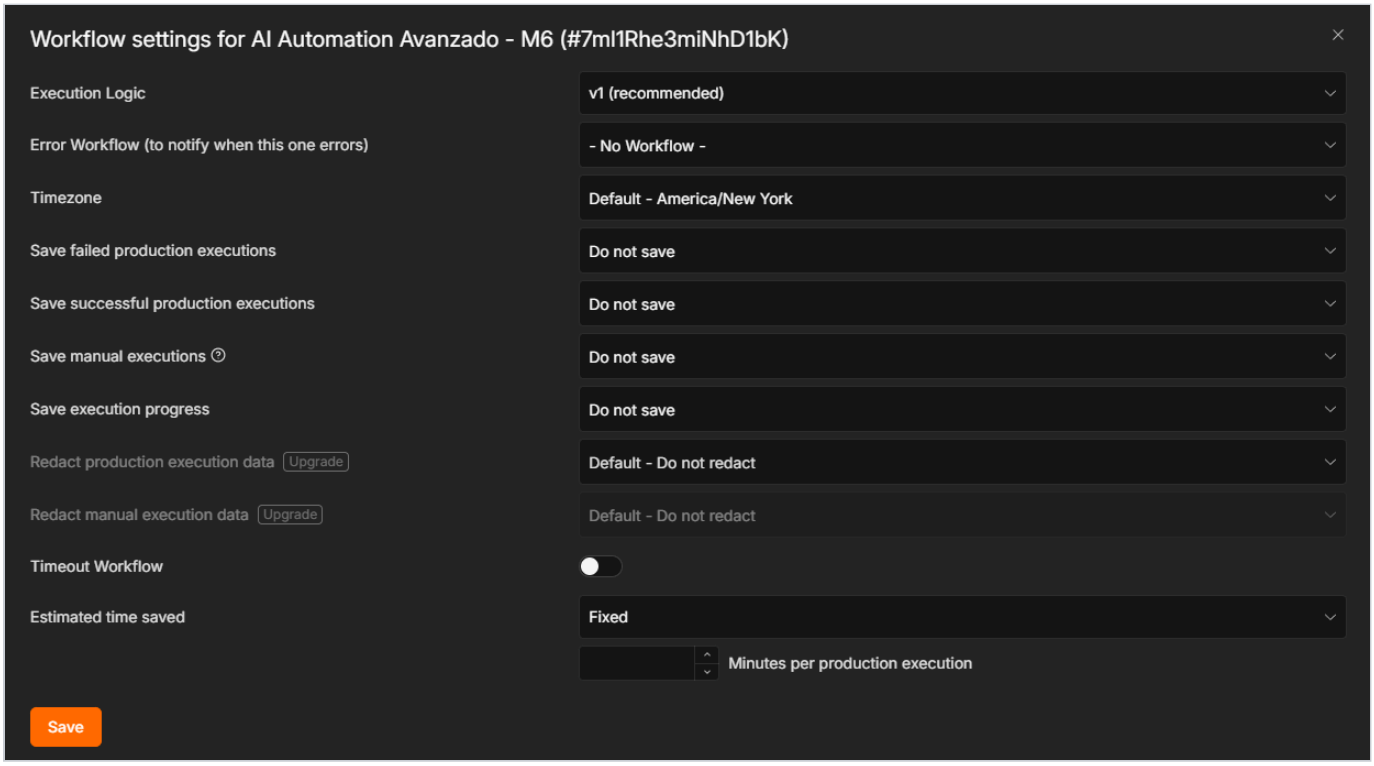
Otras contingencias, surgidas de las pruebas:

- **Email dictado.** El paciente dictó "*mwaisburd arroba gmail.com*" y Whisper lo transcribió como "*mwhisbord arroba gmail.com*", así que el turno quedó con un correo inválido. Si la nota de voz contiene un email ("arroba", "@", "punto com"), el bot responde "*Para no anotar mal tu email, ¿me lo escribís por texto?*". La regla se verificó con casos de prueba del nodo, no con un audio en vivo.
- **Mensajes repetidos.** Telegram reintentó tres veces la entrega del mismo mensaje (mismo `update_id`). El nodo "Filtrar repetidos" procesa cada `update_id` una sola vez.
- **Fuera de alcance.** Una pregunta ajena a la clínica respondía "Listo." (heredado del M5); ahora responde "*Solo puedo ayudarte con turnos e información de la Clínica LoDeTincho. Esa consulta no la puedo responder.*".

7. Compliance: audios en memoria volátil y destrucción

Diseño: las notas de voz de los pacientes y los audios sintetizados existen solo mientras corre la ejecución; ninguna queda guardada en n8n ni sale del flujo.

1. **El workflow no guarda ejecuciones.** En *Workflow settings*: ejecuciones exitosas, fallidas y manuales en **Do not save**, y sin guardar el progreso. n8n 2.x ya no tiene un modo de binarios "solo en memoria" (los guarda en disco durante la ejecución), pero borra esos archivos al cerrar una ejecución que no se guarda.
2. **"Descartar audio (compliance)"** es el último nodo: devuelve solo metadata (`enviado`, `chat_id`, `audio_destruido: true`), sin binario.
3. **Ningún audio se sube** a Drive, Airtable ni a otro almacenamiento: la memoria del paciente guarda el resumen del texto, nunca el audio.
4. **La nota roja del lienzo** agrupa los nodos de salida y de compliance (ver captura de la pieza 2).



Verificación. Con los settings aplicados se ejecutó una consulta y se comprobó que n8n no registró la ejecución ni creó una carpeta nueva de binarios en el almacenamiento de n8n.

Datos sensibles del canal. La clave del bot y las API keys viven en las credenciales cifradas de n8n; los JSON exportados del workflow no las contienen.

8. Pruebas de punta a punta y hallazgos

Pruebas con notas de voz reales desde Telegram (11/10/2026), leídas de las ejecuciones de n8n:

#	Nota de voz (transcripción de Whisper)	Ruta	Respuesta hablada	Resultado
308	"Y agéndame otro para una tomografía para el 11 de octubre a las 21.30."	CREAR_TURNO → Worker 1 → planilla 06	"Hola Juan Carlos, tu turno de tomografía para el 11/10/2026 a las 21:30 quedó Asignado." (87 car., audio 6 s)	✅ turno registrado y confirmado por voz
321	"¿Me decís que profesionales atienden cardiología?"	INFO → Agente RAG → validador (aprobada)	"Cardiología tiene tres profesionales. Dr. Fairman Enrique, Dra. Carolina Granada Ayala y Dr. Flavio Delfino. ¿Querés turno con alguno?" (134 car., audio 10 s)	✅ dato de la base documental, con cita en el texto
323	"Sí, con Delfino para mañana a las 11 y 22."	turno (repregunta con memoria)	confirmación de 89 caracteres, audio 6 s	✅ usó el contexto de la charla
320	(silencio) → "Subtítulos realizados por la comunidad de Amara.org"	contingencia	aviso por texto	✅ alucinación detectada
331	"¿Y el precio del dólar blue ahora?"	FUERA_DE_ALCANCE	mensaje de límite (106 car.)	✅
302	Dicho: "Mi email es mwaisburd arroba gmail.com" → Whisper: "Mi email es mwhisbord arroba gmail.com"	turno	—	❌ email mal transcripto → se agregó "pedir email por escrito"

Hallazgos

- **Latencia:** de 18 a 21 s por pedido completo, contra el objetivo de 6–8 s de la unidad 4. Whisper tarda ~2 s y ElevenLabs ~2,5 s; el resto es el Manager heredado (Router, RAG, validador, memoria, logs de Slack y Gmail en serie). Próxima mejora: mandar el audio antes de los logs y la memoria.
- **Whisper alucina ante silencio** (pieza 6): la contingencia no puede limitarse a "texto vacío".
- **Los datos exactos no van por voz:** emails y listas largas se piden o entregan por escrito.
- **Telegram reintenta entregas** si el webhook demora: hace falta filtrar por `update_id`.

9. Anexo: configuración exportada en formato texto

Extracto del workflow exportado de n8n (`AI Automation Avanzado - M6.json`): los nodos de la capa de voz con su tipo y parámetros, más los *settings* del workflow. El JSON completo (82 nodos) y los de los sub-workflows están versionados en el repositorio del proyecto (`Avanzado/Modulo6/`). Las credenciales solo aparecen por nombre: los secretos quedan cifrados en n8n.

Settings del workflow

```
{
  "executionOrder": "v1",
  "binaryMode": "separate",
  "saveExecutionProgress": false,
  "saveManualExecutions": false,
  "saveDataErrorExecution": "none",
  "saveDataSuccessExecution": "none"
}
```

Mensaje de Telegram

```
{
  "type": "n8n-nodes-base.telegramTrigger",
  "typeVersion": 1.2,
  "parameters": {
    "updates": [
      "message"
    ],
    "additionalFields": {}
  },
  "credentials": {
    "telegramApi": "Telegram account"
  }
}
```

Filtrar repetidos

```
{
  "type": "n8n-nodes-base.code",
  "typeVersion": 2,
  "parameters": {
    "jsCode": "// Telegram reintenta la entrega si el webhook tarda en responder: el mismo update
llega varias veces.\n// Se procesa cada update_id una sola vez (memoria del workflow; persiste con el
workflow publicado).\nconst s = $getWorkflowStaticData('global');\ns.vistos = s.vistos || [];\nconst
id = $json.update_id;\nif (s.vistos.includes(id)) return [];\ns.vistos.push(id);\ns.vistos =
s.vistos.slice(-200);\nreturn [$input.first()];"
  }
}
```

¿Nota de voz?

```
{
  "type": "n8n-nodes-base.if",
  "typeVersion": 2,
  "parameters": {
    "conditions": {
      "options": {
        "caseSensitive": true,
        "leftValue": "",
        "typeValidation": "loose",
        "version": 1
```

```
      },
      "conditions": [
        {
          "id": "7fab2622-31e3-4a28-9cc8-3a38ac1f1e85",
          "leftValue": "={{ $json.message.voice || $json.message.audio ? 'si' : 'no' }}",
          "operator": {
            "type": "string",
            "operation": "equals"
          },
          "rightValue": "si"
        }
      ],
      "combinator": "and"
    },
    "options": {}
  }
}
```

Descargar nota de voz

```
{
  "type": "n8n-nodes-base.telegram",
  "typeVersion": 1.2,
  "parameters": {
    "resource": "file",
    "fileId": "={{ $json.message.voice ? $json.message.voice.file_id : $json.message.audio.file_id
}}",
    "additionalFields": {}
  },
  "credentials": {
    "telegramApi": "Telegram account"
  }
}
```

Transcribir (Whisper)

```
{
  "type": "@n8n/n8n-nodes-langchain.openAi",
  "typeVersion": 2.3,
  "parameters": {
    "resource": "audio",
    "operation": "transcribe",
    "binaryPropertyName": "data",
    "options": {
      "language": "es",
      "temperature": 0
    }
  },
  "credentials": {
    "openAiApi": "OpenAI account"
  }
}
```

Evaluar transcripción

```
{
  "type": "n8n-nodes-base.code",
  "typeVersion": 2,
  "parameters": {
    "jsCode": "// Contingencia de audio: Whisper no devuelve vacío ante silencio o ruido, ALUCINA
frases de subtítulos\n// (p. ej. \"Subtítulos realizados por la comunidad de Amara.org\"). Se marca
como ilegible:\n// texto vacío o < 3 letras, solo signos, o una frase alucinada conocida.\nconst
texto = String($json.text || '').trim();\nconst letras = texto.replace(/[^\\p{L}\\p{N}]/gu,
'');\nconst ALUCINACIONES = [\n  /amara\\.org/i, /subt[ií]tulos (realizados|hechos|por)/i, /gracias
```

```
por ver/i, /suscr[ií]bete/i,\n  /^\\W*(m[uú]sica|aplausos|risas|silencio)\\W*$/i, /^\\W*gracias\\.?
\\W*$/i, /www\\.\\S+/i\n];\nconst alucinada = ALUCINACIONES.some(r => r.test(texto));\nconst ilegible
= letras.length < 3 || alucinada;\n// Un email dictado por voz se transcribe mal (\"mwhisbord arroba
gmail punto com\"): se pide por escrito.\nconst dicta_email = !ilegible &&
(/\\barroba\\b/i.test(texto) || /@/.test(texto) || /punto\\s*com\\b/i.test(texto));\nreturn [{ json:
{ ...$json, text: texto, ilegible, motivo_ilegible: ilegible ? (alucinada ? 'alucinación de Whisper'
: 'audio vacío') : '', dicta_email } }];"
  }
}
```

¿Audio vacío o ilegible?

```
{
  "type": "n8n-nodes-base.if",
  "typeVersion": 2,
  "parameters": {
    "conditions": {
      "options": {
        "caseSensitive": true,
        "leftValue": "",
        "typeValidation": "loose",
        "version": 1
      },
      "conditions": [
        {
          "id": "af5c353f-69e8-4678-8a37-72a6160e1abc",
          "leftValue": "={{ $json.ilegible }}",
          "operator": {
            "type": "boolean",
            "operation": "true",
            "singleValue": true
          }
        }
      ],
      "combinator": "and"
    },
    "options": {}
  }
}
```

Avisar: no se escuchó

```
{
  "type": "n8n-nodes-base.telegram",
  "typeVersion": 1.2,
  "parameters": {
    "chatId": "={{ $('Mensaje de Telegram').first().json.message.chat.id }}",
    "text": "Perdón, no logré escuchar tu audio con claridad. ¿Podés grabarlo de nuevo en un lugar
con menos ruido, o escribirme tu consulta?",
    "additionalFields": {
      "appendAttribution": false
    }
  },
  "credentials": {
    "telegramApi": "Telegram account"
  }
}
```

¿Dictó un email?

```
{
  "type": "n8n-nodes-base.if",
  "typeVersion": 2,
  "parameters": {
```

```
    "conditions": {
      "options": {
        "caseSensitive": true,
        "leftValue": "",
        "typeValidation": "loose",
        "version": 1
      },
      "conditions": [
        {
          "id": "f9cbc31f-3c9b-4794-bbac-8caba8c46e00",
          "leftValue": "={{ $json.dicta_email }}",
          "operator": {
            "type": "boolean",
            "operation": "true",
            "singleValue": true
          }
        }
      ],
      "combinator": "and"
    },
    "options": {}
  }
}
```

Pedir email por escrito

```
{
  "type": "n8n-nodes-base.telegram",
  "typeVersion": 1.2,
  "parameters": {
    "chatId": "={{ $('Mensaje de Telegram').first().json.message.chat.id }}",
    "text": "Para no anotar mal tu email, ¿me lo escribís por texto? Por audio se confunden letras.",
    "additionalFields": {
      "appendAttribution": false
    }
  },
  "credentials": {
    "telegramApi": "Telegram account"
  }
}
```

Entrada Telegram

```
{
  "type": "n8n-nodes-base.code",
  "typeVersion": 2,
  "parameters": {
    "jsCode": "// Une las dos ramas de Telegram (voz transcrita o texto) en el formato del
Manager.\nconst t = $('Mensaje de Telegram').first().json.message || {};\nconst esVoz = !!(t.voice ||
t.audio);\nconst texto = esVoz ? String($json.text || '').trim() : String(t.text ||
'').trim();\nreturn [{ json: {\n  canal: esVoz ? 'telegram_voz' : 'telegram_texto',\n  sessionId:
'tg-' + t.chat.id,\n  tg_chat_id: t.chat.id,\n  mensaje: texto,\n  nombre_remitente:
[t.from?.first_name, t.from?.last_name].filter(Boolean).join(' ')\n} }];"
  }
}
```

¿Vino por Telegram?

```
{
  "type": "n8n-nodes-base.if",
  "typeVersion": 2,
  "parameters": {
    "conditions": {
      "options": {
```

```
        "caseSensitive": true,
        "leftValue": "",
        "typeValidation": "loose",
        "version": 1
      },
      "conditions": [
        {
          "id": "e671822e-eb15-4f8e-9d9d-08f07280baeb",
          "leftValue": "={{ $('Normalizar entrada').first().json.canal }}",
          "operator": {
            "type": "string",
            "operation": "startsWith"
          },
          "rightValue": "telegram"
        }
      ],
      "combinator": "and"
    },
    "options": {}
  }
}
```

Preparar respuesta hablada

```
{
  "type": "n8n-nodes-base.code",
  "typeVersion": 2,
  "parameters": {
    "jsCode": "// Respuesta para el canal de voz: sin citas, links, emojis ni markdown, y como máximo
200 caracteres\n// (ElevenLabs cobra por carácter; la regla también está en el System Prompt del
Agente RAG).\nconst MAX = 200;\nconst original = String($json.output || '');\nlet t = original\n
.replace(/\\[Fuente:[^\\]]*\\]/gi, '')          // citas de fuente (quedan en el texto, no en el
audio)\n  .replace(/https?:\\/\\/\\S+/g, '')              // URLs\n  .replace(/[*_`#>]/g, '')
// markdown\n  .replace(/^\\s*[-•]\\s+/gm, '')                // viñetas\n  .replace(/[\\u{1F300}-
\\u{1FAFF}\\u{2600}-\\u{27BF}\\u{FE0F}]/gu, '') // emojis\n  .split(/\\s*\\n+\\s*/).map(s =>
s.trim()).filter(Boolean)\n  // cada línea se une como frase: si no termina en signo, se le agrega un
punto (\"atienden:\" -> \"atienden:\")\n  .map(s => /[.?!:;,]$/.test(s) ? s : s + '.').join(' ')\n
.replace(/\\s{2,}/g, ' ').trim();\nif (t.length > MAX) {\n  // cortar en el último fin de oración (.
? !) que entre en 200; si no hay, en la última palabra\n  const corte = t.slice(0, MAX + 1);\n  //
fines de oración reales: no cuentan las abreviaturas (Dr., Dra., Sr., Sra., Lic., etc.)\n  let fin =
-1;\n  for (const m of corte.matchAll(/[.?!](?=\\s|$)/g)) {\n    const previa = corte.slice(0,
m.index).split(/\\s/).pop();\n    if (m[0] === '.' &&
/^(Dr|Dra|Sr|Sra|Lic|Dres|Av|Nro|Ing)$/i.test(previa)) continue;\n    fin = m.index;\n  }\n  t = (fin
> 60 ? corte.slice(0, fin + 1) : corte.slice(0, corte.lastIndexOf(' ')) + '…').trim();\n}\nconst n =
$('Normalizar entrada').first().json;\nreturn [{ json: { texto_voz: t, caracteres: t.length,
texto_completo: original, canal: n.canal, tg_chat_id: n.tg_chat_id } }];"
  }
}
```

¿Responder con voz?

```
{
  "type": "n8n-nodes-base.if",
  "typeVersion": 2,
  "parameters": {
    "conditions": {
      "options": {
        "caseSensitive": true,
        "leftValue": "",
        "typeValidation": "loose",
        "version": 1
      },
      "conditions": [
        {
          "id": "31b9608a-f820-4d90-90d0-92cec23f7f2b",
```

```
          "leftValue": "={{ $json.canal }}",
          "operator": {
            "type": "string",
            "operation": "equals"
          },
          "rightValue": "telegram_voz"
        }
      ],
      "combinator": "and"
    },
    "options": {}
  }
}
```

Sintetizar voz (ElevenLabs)

```
{
  "type": "@elevenlabs/n8n-nodes-elevenlabs.elevenLabs",
  "typeVersion": 1,
  "parameters": {
    "resource": "speech",
    "operation": "textToSpeech",
    "voice": {
      "__rl": true,
      "mode": "id",
      "value": "pFZP5JQG7iQjIQuC4Bku"
    },
    "text": "={{ $json.texto_voz }}",
    "additionalOptions": {
      "model": {
        "__rl": true,
        "mode": "list",
        "value": "eleven_multilingual_v2"
      },
      "outputFormat": "mp3_44100_128",
      "languageCode": "es",
      "voiceSettings": "{\n  \"stability\": 0.55,\n  \"similarity_boost\": 0.75,\n  \"style\": 0,\n
\"use_speaker_boost\": true,\n  \"speed\": 1\n}"
    }
  },
  "credentials": {
    "elevenLabsApi": "ElevenLabs account"
  }
}
```

Enviar nota de voz (Telegram)

```
{
  "type": "n8n-nodes-base.telegram",
  "typeVersion": 1.2,
  "parameters": {
    "operation": "sendAudio",
    "chatId": "={{ $('Preparar respuesta hablada').first().json.tg_chat_id }}",
    "binaryData": true,
    "binaryPropertyName": "data",
    "additionalFields": {
      "title": "AgendaBot",
      "fileName": "respuesta.mp3"
    }
  },
  "credentials": {
    "telegramApi": "Telegram account"
  }
}
```

Descartar audio (compliance)

```
{
  "type": "n8n-nodes-base.code",
  "typeVersion": 2,
  "parameters": {
    "jsCode": "// Compliance: el audio del paciente y el sintetizado no se conservan.\n// Se devuelve solo metadata (sin binario); además el workflow no guarda ejecuciones,\n// así que n8n borra los archivos temporales de la ejecución al terminar.\nreturn $input.all().map(i => ({ json: { enviado: true, chat_id: i.json.result?.chat?.id ?? null, audio_destruido: true } }));"
  }
}
```

Enviar texto (Telegram)

```
{
  "type": "n8n-nodes-base.telegram",
  "typeVersion": 1.2,
  "parameters": {
    "chatId": "={{ $json.tg_chat_id }}",
    "text": "={{ $json.texto_completo }}",
    "additionalFields": {
      "appendAttribution": false
    }
  },
  "credentials": {
    "telegramApi": "Telegram account"
  }
}
```